Culture, collaboration Git et conteneurs

Mastère ESI

Réunir le travail d'une équipe sans casser ce qui marche

DevOps · Séquence 1

Objectifs

- Situer chaque étape du cycle DevOps et son responsable.
- Intégrer le travail de plusieurs personnes par branches et pull requests.
- Relire le code d'un pair et argumenter une demande de modification.
- Résoudre un conflit de fusion en expliquant la version retenue.
- Démarrer l'application et sa base avec Docker Compose.

Prérequis

- Ligne de commande et bases de Git : `clone`, `add`, `commit`, `push`.
- Un compte GitHub, invité sur le dépôt `voyages-scolaires`.
- Les trois branches produites le matin en Laravel.

Sommaire

Partie	Contenu
1	Culture DevOps et diagnostic
2	Collaborer avec Git et GitHub : PR, revue, conflits
3	Conteneuriser l'application avec Docker Compose

1. Culture DevOps et diagnostic

Le cycle, rappel de la présentation du module

DevOps réunit ceux qui **développent** une application et ceux qui **l'exploitent** autour d'une même chaîne, automatisée autant que possible, et d'une responsabilité partagée sur le résultat en production.

Étape	Question	Outil dans ce module
Coder	Comment travailler à plusieurs sur le même code ?	Git, GitHub, pull requests
Construire	L'application tourne-t-elle partout de la même façon ?	Docker, Docker Compose
Tester	Une modification casse-t-elle l'existant ?	Tests Laravel, GitHub Actions
Déployer	Comment livrer sans geste manuel risqué ?	GitHub Actions, Kubernetes (k3d)
Décrire l'infrastructure	Comment recréer un environnement à l'identique ?	Terraform
Surveiller	Comment savoir qu'elle tombe avant les utilisateurs ?	Journaux, contrôle de disponibilité, alerte

Tout commence par la première ligne : **une chaîne automatique ne vaut que si le code qui y entre est intégré proprement.** C'est l'objet de la partie 2.

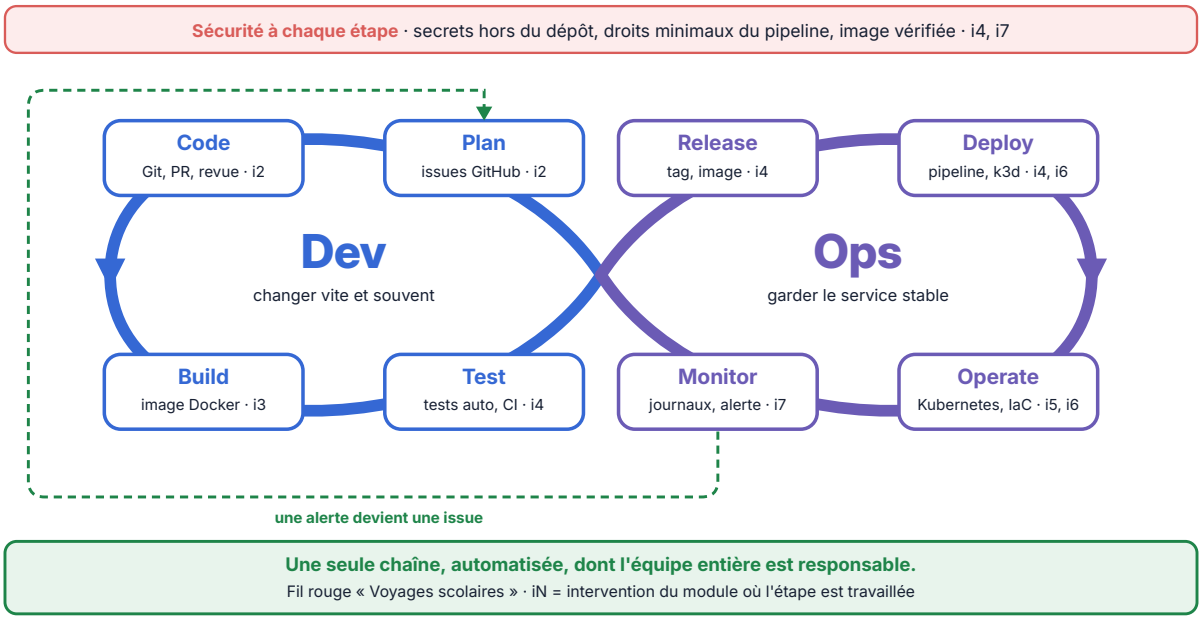


Fig. 1 - Cycle DevOps appliqué au fil rouge : quatre étapes côté développement, quatre côté exploitation, une boucle de rétroaction de la surveillance vers la planification ; iN indique l'intervention du module.

Pourquoi DevOps

Développement et exploitation ont longtemps eu des objectifs opposés : livrer du nouveau d'un côté, éviter les pannes de l'autre. Le code était « jeté par-dessus le mur » ; les livraisons, rares, devenaient grosses et risquées. Le mouvement DevOps naît en 2009 (conférence Velocity, premiers *DevOpsDays* à Gand) pour réunir les deux métiers autour d'une chaîne commune. Ses piliers se résument par **CALMS** : culture de responsabilité partagée, automatisation, petits lots livrés souvent (*lean*), mesure, partage des outils et des connaissances.

Trois niveaux d'automatisation

Pratique	Ce qui est automatique	Ce qui reste humain
Intégration continue (CI)	Chaque modification fusionnée est construite et testée	La revue de code
Livraison continue	Chaque version validée est prête à partir en production	La décision de livrer
Déploiement continu (CD)	Chaque version validée part en production	Rien dans la chaîne ; la surveillance prend le relais

Mesurer : les quatre indicateurs DORA

Indicateur	Mesure	Famille
Fréquence de déploiement	Nombre de livraisons en production sur une période	Rapidité
Délai de mise en production	Temps entre un commit et sa présence en production	Rapidité
Taux d'échec des changements	Part des livraisons qui provoquent un incident	Stabilité
Temps de rétablissement	Durée pour revenir à un service normal après un incident	Stabilité

Les études DORA (*Accelerate*, 2018) montrent que les équipes les plus performantes progressent **sur les deux familles à la fois** : livrer souvent, par petits lots, rend chaque livraison moins risquée.

Exercices

Exercice 1 — Placer sur le cycle. Seul, sur papier. Pour chaque action, donnez l'étape du cycle et dites si elle relève de Dev, d'Ops ou des deux.

Action	Étape	Dev, Ops, les deux
a. Ouvrir l'issue « Afficher le prix en euros »		
b. Fusionner une PR après revue		
c. Construire l'image Docker de l'application		
d. Lancer `php artisan test` à chaque push		
e. Étiqueter la version `v0.1.0`		
f. Appliquer le manifeste Kubernetes de la nouvelle version		
g. Redémarrer automatiquement un conteneur tombé		
h. Recevoir une alerte quand `/api/voyages` ne répond plus		

Exercice 2 — Analyser l'incident du vendredi. Seul, puis confrontation à trois.

> Vendredi 17 h 30, le développeur dépose par FTP la nouvelle version de « Voyages scolaires » et part en week-end. Il a oublié d'exécuter une migration : la liste des voyages renvoie une erreur 500. Personne ne s'en aperçoit avant lundi 9 h, quand les parents appellent le lycée. Il faut ensuite trois heures pour retrouver quelle version tournait vendredi midi et la remettre en ligne.

Relevez au moins trois défaillances. Pour chacune : la pratique DevOps qui l'aurait évitée, l'étape du cycle concernée et l'indicateur DORA dégradé.

Exercice 3 — La chaîne de Voyages scolaires. À trois, une seule production. Décrivez en six étapes au plus la chaîne cible du projet, de la PR à l'alerte : pour chaque étape, ce qui se passe automatiquement, l'outil, et qui est prévenu si elle échoue. Confrontez ensuite votre chaîne au déroulé du module : quelle étape aviez-vous oubliée ?

Diagnostic

Seul, sans aide, 10 minutes. Ce questionnaire n'est pas noté : il sert à adapter les prochaines interventions à ce que vous savez déjà.

1. Après `git commit`, les modifications sont enregistrées…

- [ ] sur GitHub
- [ ] dans le dépôt local
- [ ] dans la branche `main` du dépôt distant

2. Quelle commande récupère les nouveautés du dépôt distant **sans** les fusionner dans votre branche ?

- [ ] `git fetch`
- [ ] `git pull`
- [ ] `git clone`

3. Après `git switch -c feature/x` puis `git commit`, le commit appartient…

- [ ] à `main`
- [ ] à `feature/x`

- [ ] aux deux branches

4. Pourquoi le fichier `.env` figure-t-il dans `.gitignore` ?

- [ ] Pour accélérer Git
- [ ] Parce qu'il contient des secrets et des réglages propres à chaque poste
- [ ] Parce que Laravel ne le lit pas

5. Après `chmod 600 .env`, qui peut lire le fichier ?

- [ ] Son propriétaire seulement
- [ ] Son propriétaire et son groupe
- [ ] Tout le monde, sans pouvoir l'écrire

6. Quelle commande affiche en continu les nouvelles lignes d'un journal ?

- [ ] `cat storage/logs/laravel.log`
- [ ] `tail -f storage/logs/laravel.log`
- [ ] `less storage/logs/laravel.log`

7. Quelle phrase décrit le mieux la relation entre une image et un conteneur Docker ?

- [ ] Ce sont deux noms de la même chose
- [ ] Le conteneur est une exécution d'une image, qui sert de modèle
- [ ] L'image est une sauvegarde d'un conteneur arrêté

8. Que fait `docker compose up -d` ?

- [ ] Télécharge Docker
- [ ] Démarre en arrière-plan les services décrits dans le fichier Compose
- [ ] Construit une machine virtuelle

2. Collaborer avec Git et GitHub

Le problème
Ce matin, trois personnes ont travaillé sur trois branches du même dépôt. Deux d'entre elles ont modifié les **mêmes fichiers** (`routes/web.php` , `VoyageController.php` , `index.blade.php`). Il faut maintenant tout réunir dans `main` , sans perdre le travail de quiconque et sans jamais laisser `main` dans un état cassé.

Le GitHub flow

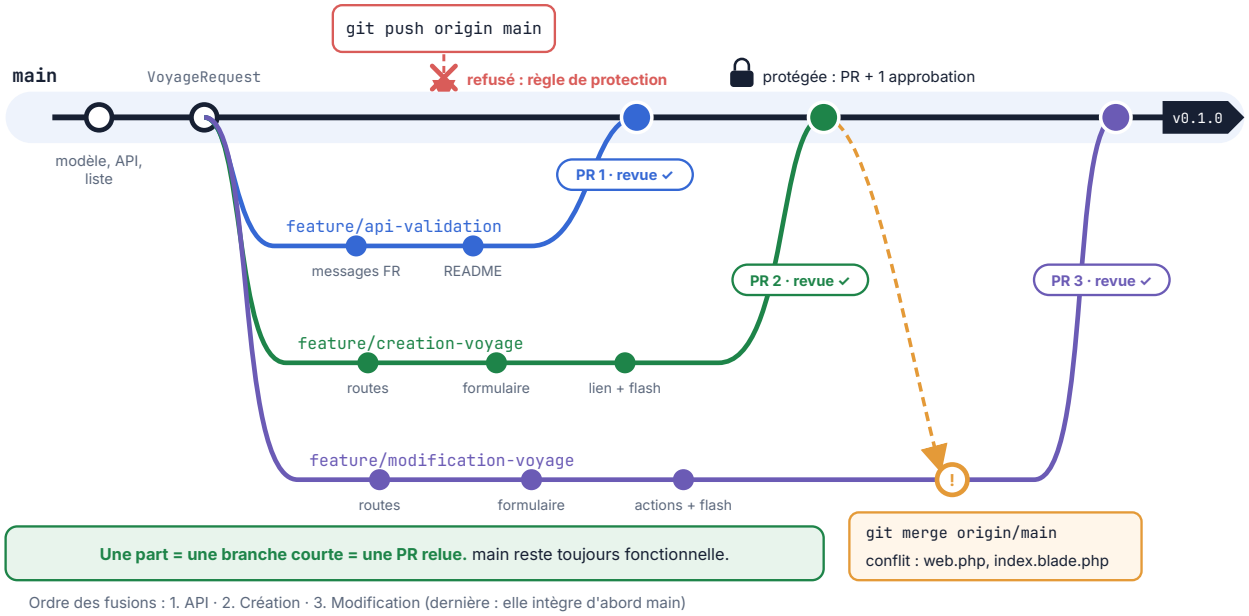


Fig. 2 - GitHub flow appliqué aux trois parts du matin : trois branches courtes partent de main, chacune revient par une pull request relue ; la dernière intègre d'abord main et résout le conflit.

Règle	Raison
`main` fonctionne toujours	C'est elle qui sera testée, construite et déployée automatiquement.
Une branche courte par changement	Moins elle vit longtemps, moins elle diverge de `main`.
Toute fusion passe par une **pull request** (PR)	La PR est le lieu de la discussion, de la revue et, bientôt, des tests automatiques.
Une approbation au moins avant fusion	Deux personnes au moins comprennent chaque ligne livrée.
La branche est supprimée après fusion	Son histoire est désormais dans `main`.

Le **Gitflow** des supports de l'an dernier ajoute une branche `develop` et des branches de version. Il convient à des livraisons espacées et numérotées ; pour une petite équipe qui livre en continu, le GitHub flow suffit et laisse moins de branches à synchroniser.

Protéger `main`

L'enseignant, propriétaire du dépôt, crée la règle en direct : **Settings → Rules → Rulesets → New branch ruleset**, cible *Default branch*, avec :

- *Restrict deletions* et *Block force pushes* ;
- *Require a pull request before merging*, *Required approvals* : 1.

Le dépôt est **public** : avec un compte GitHub gratuit, ces règles ne s'appliquent qu'aux dépôts publics. Le code ne contient aucun secret, puisque `.env` reste hors du dépôt.

Vérifiez la règle. Sur votre poste, sur `main`, faites un commit sans importance et tentez `git push`. Le refus doit mentionner une violation de règle du dépôt. Annulez ensuite ce commit local avec `git reset --hard origin/main`.

Ouvrir sa pull request

Votre branche a été publiée ce matin (`git push -u origin <branche>`). Sur GitHub, ouvrez une PR de votre branche vers `main`. Sa description suit ce modèle :

```
## Quoi
Ce que la branche ajoute, en deux ou trois phrases.

## Pourquoi
Le besoin ou le problème réglé.

## Comment tester
Les commandes et les gestes pour vérifier,
y compris une saisie invalide.
```

Relire la pull request d'un autre

Chacun relit une PR, en cercle : **API relit Création, Création relit Modification, Modification relit API.** Récupérez la branche pour la tester réellement :

```
git fetch origin
git switch feature/creation-voyage
php artisan migrate
php artisan serve
```

Dans l'onglet *Files changed* de la PR, vérifiez :

- le code fait ce qu'annonce la description, et le test décrit fonctionne ;
- vous comprenez chaque ligne, sinon vous posez la question ;
- aucun fichier n'est en trop : `.env`, `vendor/`, base SQLite, fichiers d'éditeur ;
- les noms de variables, routes et vues sont explicites et cohérents avec le reste.

Un commentaire utile tient en trois temps : **constat, raison, proposition.**

> « `store()` utilise `$request->all()` : un champ non prévu serait enregistré. Proposition : `$request->validated()`, comme dans l'API. »

Terminez par *Review changes* : **Request changes** pour au moins un point justifié, puis **Approve** quand l'auteur l'a corrigé. La correction se fait par un nouveau commit sur la même branche : la PR se met à jour toute seule.

Fusionner, puis résoudre un conflit

Ordre de fusion du premier tour : **1. API, 2. Création, 3. Modification.** Après chaque fusion, chacun met à jour son `main` local avec `git switch main` puis `git pull`.

Quand vient son tour, la PR « Modification » affiche *This branch has conflicts that must be resolved* : `main` a reçu la Création, qui a touché les mêmes lignes. On intègre `main` dans la branche, **sur son poste**, pour pouvoir tester le résultat :

```
git switch feature/modification-voyage
git fetch origin
git merge origin/main
```

```
CONFLICT (content): Merge conflict in routes/web.php
Automatic merge failed; fix conflicts and then commit the result.
```

Git a fusionné tout ce qu'il pouvait et a laissé, dans chaque fichier en conflit, les deux versions des lignes concurrentes :

```
<<<<<<< HEAD
Route::get('/voyages/{voyage}/edit', ...);
Route::put('/voyages/{voyage}', ...);
```

```
Route::delete('/voyages/{voyage}', ...);
=======
Route::get('/voyages/create', ...);
Route::post('/voyages', ...);
>>>>>>> origin/main
```

`HEAD` désigne votre branche ; la partie après `=======` vient de `main`. Résoudre, ce n'est **pas choisir un camp** : c'est écrire la version qui respecte les deux intentions. Ici, garder les cinq routes ; ailleurs, garder un seul exemplaire d'un bloc ajouté des deux côtés, comme l'affichage du message flash.

1. `git status` liste les fichiers en conflit.
2. Éditez chacun : version finale écrite, marqueurs `<<<<<<<`, `=======`, `>>>>>>>` supprimés.
3. **Testez** : `php artisan route:list --path=voyages`, puis liste, création, modification et suppression dans le navigateur.
4. `git add` des fichiers résolus, puis `git commit` pour conclure la fusion.
5. `git push` : la PR redevient fusionnable.

Lisez enfin l'histoire produite :

```
git log --oneline --graph --all
```

Second tour : de l'issue à la fusion

L'enseignant a ouvert trois issues sur le dépôt. Chacune fait l'objet d'une branche, d'une PR qui contient `Closes #<numéro>` dans sa description, et d'une revue dans le même cercle. Toutes trois enrichissent la carte d'un voyage, `resources/views/components/voyage-card.blade.php`, en ajoutant un élément `<li>` **en tête** de sa liste d'informations.

Issue	Prise par	Indication
#7 Afficher la destination sur la carte	Modification	Icône `bi-geo-alt`, `{{ $voyage->destination }}`
#8 Afficher la durée du voyage sur la carte	API	Icône `bi-hourglass-split`, méthode `dureeEnJours()` du modèle ; « 1 jour » au singulier
#9 Afficher le nombre de classes concernées sur la carte	Création	Icône `bi-mortarboard`, `classes_count` chargé par `withCount('classes')` ; vérifier aussi la page d'accueil

Ordre de fusion : **Modification, API, Création.** Les trois branches insèrent leur ligne au même endroit : à ce tour, l'API puis la Création intègrent `main` et résolvent leur conflit en gardant **toutes** les lignes. À la fin de l'après-midi, chacun en a donc résolu un.

Critère de réussite : chacun a ouvert une PR relue, demandé une modification argumentée et résolu un conflit ; `main` contient toutes les fonctions et l'application démarre depuis un clone neuf.

Restitution

Chacun, sur son poste projeté :

1. présente sa PR : description, commentaire reçu, commentaire donné ;
2. montre son conflit : les deux versions, celle retenue et pourquoi ;
3. commente `git log --oneline --graph` autour de sa fusion.

Pour finir, l'enseignant clone le dépôt dans un dossier neuf et installe l'application devant l'équipe. Comptez les commandes nécessaires et notez ce qui pourrait différer d'un poste à l'autre : version de PHP, extensions, base de données. **C'est le problème de l'intervention 3.**

Extensions

- **Annuler une fusion fautive.** Sur une branche, `git revert -m 1 <commit-de-fusion>`, puis PR. Pourquoi ne pas réécrire l'histoire de `main` à la place ?
- **Marquer une version.** `git tag -a v0.1.0 -m "Liste, API et formulaires"`, puis `git push origin v0.1.0`, et une *release* GitHub qui résume les PR fusionnées.
- **Modèle de PR.** Placez le modèle de description dans `.github/pull_request_template.md`, par une PR : GitHub le proposera à chaque nouvelle PR.

3. Conteneuriser l'application

Le problème

Le clone neuf de la partie 2 a demandé une dizaine de commandes et suppose, sur chaque poste, la bonne version de PHP, les bonnes extensions, Composer, une base et un `.env`. Un conteneur embarque le système et les dépendances de l'application : le démarrage ne dépend plus du poste.

Terme	Ce que c'est
Image	Un modèle en lecture seule, fait de couches empilées : système, extensions, dépendances, code.
Conteneur	Une instance de l'image qui s'exécute. On le supprime et on le recrée sans regret.
Registre	Le dépôt où l'on publie et récupère des images : Docker Hub, GitHub Container Registry.
Volume	Un stockage géré par Docker, qui survit à la suppression du conteneur.

Contrairement à une machine virtuelle, un conteneur n'embarque pas de système complet : il partage le noyau de l'hôte, démarre en quelques secondes et pèse bien moins lourd.

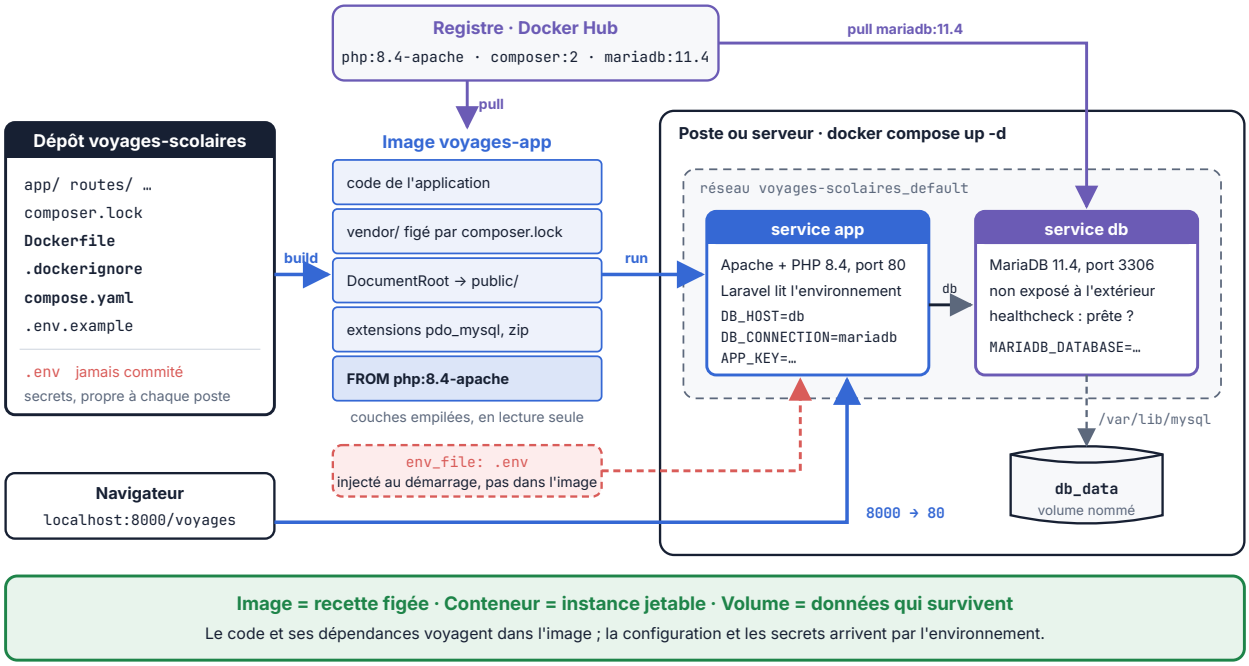


Fig. 3 - Le dépôt fournit Dockerfile et compose.yaml ; l'image de l'application est construite à partir de php:8.4-apache ; Compose lance les services app et db sur un réseau interne, la base écrit dans le volume db_data, le .env est injecté au démarrage.

Vérifier l'installation

```
docker --version
docker compose version
docker run --rm hello-world
```

La dernière commande télécharge une image, lance un conteneur qui affiche un message, puis le supprime (`--rm`). Si elle échoue, Docker Desktop n'est pas démarré.

Le Dockerfile

À la racine du dépôt, le fichier `Dockerfile` décrit la construction de l'image :

```
FROM php:8.4-apache

RUN apt-get update \
    && apt-get install -y --no-install-recommends git unzip libzip-dev \
    && docker-php-ext-install pdo_mysql zip \
    && a2enmod rewrite \
    && rm -rf /var/lib/apt/lists/*

ENV APACHE_DOCUMENT_ROOT=/var/www/html/public
RUN sed -ri -e 's!/var/www/html!${APACHE_DOCUMENT_ROOT}!g' \
    /etc/apache2/sites-available/*.conf
RUN sed -ri -e 's!/var/www/!${APACHE_DOCUMENT_ROOT}!g' \
    /etc/apache2/apache2.conf /etc/apache2/conf-available/*.conf

COPY --from=composer:2 /usr/bin/composer /usr/bin/composer
WORKDIR /var/www/html

COPY composer.json composer.lock ./
RUN composer install --no-dev --no-scripts --no-autoloader --prefer-dist

COPY . .
RUN composer dump-autoload --optimize \
    && chown -R www-data:www-data storage bootstrap/cache
```

Instruction	Rôle
`FROM php:8.4-apache`	Part de l'image officielle PHP 8.4 avec Apache, à une version écrite. Elle doit satisfaire `composer.lock` : les composants Symfony 8 de Laravel 13 exigent PHP 8.4, et `composer install` échouerait avec `php:8.3-apache`.
`RUN … docker-php-ext-install`	Ajoute les extensions PHP nécessaires : `pdo_mysql` pour MariaDB, `zip` pour Composer.
`a2enmod rewrite`	Active la réécriture d'URL, nécessaire aux routes Laravel.
`APACHE_DOCUMENT_ROOT` et `sed`	Fait servir le dossier `public/` : le reste du projet n'est pas accessible par le web.
`COPY --from=composer:2`	Récupère l'exécutable Composer depuis son image officielle.
`COPY composer.json composer.lock` puis `composer install`	Installe exactement les versions verrouillées. Copiés **avant** le code, ces fichiers permettent à Docker de réutiliser la couche `vendor/` tant qu'ils ne changent pas.
`COPY . .`	Copie le code, sauf ce que `.dockerignore` exclut.
`chown`	Laisse Apache écrire dans `storage/` et `bootstrap/cache/`.

À côté, le fichier `.dockerignore` liste ce qui ne doit pas entrer dans l'image :

```
.git
.env
vendor
node_modules
database/*.sqlite
```

compose.yaml

Le fichier `compose.yaml` décrit les deux services et leur liaison :

```yaml
services:
  app:
    build: .
    ports:
      - "8000:80"
    env_file: .env
    environment:
      DB_CONNECTION: mariadb
      DB_HOST: db
    depends_on:
      db:
        condition: service_healthy

  db:
    image: mariadb:11.4
    environment:
      MARIADB_DATABASE: ${DB_DATABASE}
      MARIADB_USER: ${DB_USERNAME}
      MARIADB_PASSWORD: ${DB_PASSWORD}
      MARIADB_RANDOM_ROOT_PASSWORD: "1"
```

```
    volumes:
      - db_data:/var/lib/mysql
    healthcheck:
      test: ["CMD", "healthcheck.sh", "--connect", "--innodb_initialized"]
      interval: 5s
      retries: 10

volumes:
  db_data:
```

- `"8000:80"` : le port 8000 de l'hôte mène au port 80 du conteneur `app`. La base n'a pas de `ports` : elle n'est joignable que depuis le réseau interne.
- `DB_HOST: db` : sur ce réseau, chaque service est joignable **par son nom**.
- `env_file: .env` : les variables du `.env` sont injectées au démarrage. Une variable d'environnement l'emporte sur le fichier `.env` lu par Laravel : `DB_CONNECTION` et `DB_HOST` remplacent donc les valeurs SQLite du poste.
- `${DB_DATABASE}` : Compose lit aussi le `.env` placé à côté de `compose.yaml` pour compléter le fichier. Un seul `.env` configure ainsi Laravel et MariaDB.
- `depends_on` avec `service_healthy` : `app` attend que la base réponde vraiment, pas seulement que son conteneur existe.

> **Attention**
>
> MariaDB n'utilise `MARIADB_DATABASE`, `MARIADB_USER` et `MARIADB_PASSWORD` qu'au **premier** démarrage, sur un volume vide. Changer le mot de passe ensuite ne change rien : il faut supprimer le volume (`docker compose down -v`), et donc les données.

Démarrer

Dans votre `.env` (jamais commité), décommentez et renseignez les variables de base, par exemple `DB_DATABASE=voyages`, `DB_USERNAME=voyages` et un mot de passe. Reportez ces noms, **sans valeur secrète**, dans `.env.example`. Puis :

```
docker compose up -d --build
docker compose ps
docker compose exec app php artisan migrate --seed
```

Ouvrez `http://localhost:8000/voyages` et `http://localhost:8000/api/voyages`.

Commande	Effet
`docker compose logs -f app`	Suit les journaux d'Apache et de Laravel.
`docker compose exec app sh`	Ouvre un terminal dans le conteneur `app`.
`docker compose down`	Arrête et supprime les conteneurs ; le volume `db_data` est conservé.
`docker compose down -v`	Supprime aussi les volumes : **les données sont perdues**.

Symptôme	Premier contrôle
`port is already allocated`	Un `php artisan serve` tourne encore ? Sinon, `"8001:80"`.
Page 500, `No application encryption key`	`APP_KEY` est-il renseigné dans `.env` ?
Erreur sur la table `sessions`	Les migrations ont-elles été lancées dans le conteneur ?
`Connection refused` vers la base	`DB_HOST=db` ; `docker compose ps` montre-t-il `db` *healthy* ?
`Access denied for user`	Identifiants changés après le premier démarrage : `down -v`.
`Permission denied` dans `storage/`	Le `chown` du Dockerfile a-t-il été exécuté ?

À trois : conteneuriser par pull request

Chacun crée une branche depuis `main` à jour, ouvre une PR et relit celle de son voisin, comme en partie 2. Les trois PR sont nécessaires pour que la pile démarre : fusionnez-les, puis testez.

1. **API** : `Dockerfile` et `.dockerignore`. Construisez l'image seule avec `docker build -t voyages-app .` et expliquez dans la PR chaque instruction.
2. **Création** : `compose.yaml` et `.env.example` complété des variables de base. Vérifiez la syntaxe avec `docker compose config`, **sans** coller la sortie dans la PR : elle contient les valeurs du `.env`.
3. **Modification** : section « Démarrer avec Docker » du `README.md` (prérequis, commandes, adresse, arrêt). Après les fusions, clonez le dépôt dans un dossier neuf et suivez le README à la lettre.

Critère de réussite : depuis un clone neuf, `cp .env.example .env`, la clé, les variables de base, `docker compose up -d --build` et la migration suffisent à afficher la liste et l'API.

Questions

1. Après `docker compose down` puis `up -d`, les voyages sont-ils toujours là ? Et après `down -v` ? Expliquez.
2. Pourquoi l'URL de la base est-elle `db` et non `127.0.0.1` dans le conteneur `app` ?
3. On remplace `FROM php:8.4-apache` par `FROM php:apache`. Quel risque cela fait-il courir à la reproductibilité ?
4. Pourquoi copier `composer.lock` avant le reste du code ? Observez la durée d'un second `docker compose build` après avoir modifié une vue.
5. Citez deux pratiques du `Dockerfile` qui garantissent que deux postes obtiennent le même résultat.

Extensions

- **Utilisateur non privilégié.** Recherchez pourquoi un conteneur qui s'exécute en `root` est un risque, et ce que fait déjà l'image `php:apache` pour les processus Apache.
- **Épingler par empreinte.** `docker images --digests` affiche l'empreinte `sha256` d'une image ; `FROM php:8.4-apache@sha256:…` fige l'image au bit près. Quel inconvénient pour les correctifs de sécurité ?

Sources

Images officielles vérifiées le 19 octobre 2026 dans la documentation Docker : php, variante Apache et mariadb. Règles de protection vérifiées le 18 octobre 2026 dans la documentation GitHub : rulesets et résolution d'un conflit en ligne de commande.

Corrigés

Mastère ESI

Réunir le travail d'une équipe sans casser ce qui marche

DevOps · Séquence 1

Diagnostic

1. Dans le dépôt local : `push` les publie. 2. `git fetch`. 3. `feature/x`. 4. Secrets et réglages propres au poste. 5. Le propriétaire seulement (`6` = lecture + écriture).
2. `tail -f`. 7. Le conteneur est une exécution de l'image. 8. Démarre les services en arrière-plan.

Exercice 1

a. Plan (les deux). b. Code (Dev, règle commune). c. Build (Dev, Dockerfile ; Ops, base de l'image). d. Test (Dev écrit, la CI exécute). e. Release (les deux). f. Deploy (Ops, par le pipeline). g. Operate (Ops, configuré par l'orchestrateur). h. Monitor (Ops reçoit, Dev corrige).

Exercice 2

Défaillance	Pratique	Étape	Indicateur DORA
Copie FTP et migration oubliée	Déploiement automatisé qui exécute les migrations	Deploy	Taux d'échec des changements
Aucun test avant la mise en ligne	Tests automatiques bloquants en CI	Test	Taux d'échec des changements
Panne découverte par les utilisateurs	Contrôle de disponibilité et alerte	Monitor	Temps de rétablissement
Version en ligne introuvable	Versions étiquetées, images versionnées, retour arrière	Release	Temps de rétablissement
Livraison isolée un vendredi soir	Petites livraisons fréquentes, en journée, à plusieurs	Plan / culture	Fréquence de déploiement

La question « qui est responsable ? » n'appelle pas un nom : c'est la chaîne qui a laissé passer l'erreur. Un bilan d'incident sans recherche de coupable cherche ce qu'il faut automatiser.

Exercice 3

Exemple de chaîne : PR ouverte ⟶ CI : tests et analyse (auteur prévenu) ⟶ fusion après revue ⟶ construction et publication de l'image étiquetée (équipe prévenue en cas d'échec) ⟶ déploiement sur le cluster avec migrations (retour à la version précédente si le contrôle de santé échoue) ⟶ surveillance de `/api/voyages` avec alerte à l'équipe. Oubli fréquent : la gestion des secrets du pipeline et le retour arrière.

Conflit du premier tour

`routes/web.php` après résolution :

```
Route::get('/voyages', [VoyageController::class, 'index'])
    ->name('voyages.index');
```

```
Route::get('/voyages/create', [VoyageController::class, 'create'])
    ->name('voyages.create');
Route::post('/voyages', [VoyageController::class, 'store'])
    ->name('voyages.store');
Route::get('/voyages/{voyage}/edit', [VoyageController::class, 'edit'])
    ->name('voyages.edit');
Route::put('/voyages/{voyage}', [VoyageController::class, 'update'])
    ->name('voyages.update');
Route::delete('/voyages/{voyage}', [VoyageController::class, 'destroy'])
    ->name('voyages.destroy');
```

Ces six routes peuvent se résumer en `Route::resource('voyages', VoyageController::class)->except('show');` : une simplification à proposer par une PR distincte, pas au milieu d'une résolution de conflit.

Dans `VoyageController.php`, garder les cinq méthodes. Dans `index.blade.php`, garder le lien « Nouveau voyage », la colonne d'actions et **un seul** bloc d'affichage du message flash.

Second tour

Le conflit de l'API, puis celui de la Création, a la même forme : deux `<li>` insérés au même endroit. On garde les deux, dans un ordre lisible. Début de la liste après les trois fusions :

```
<li class="d-flex align-items-center gap-2 mb-1">
    <i class="bi bi-geo-alt text-primary"></i>
    {{ $voyage->destination }}
</li>
<li class="d-flex align-items-center gap-2 mb-1">
    <i class="bi bi-hourglass-split text-primary"></i>
    {{ $voyage->dureeEnJours() }} {{ $voyage->dureeEnJours() > 1 ? 'jours' : 'jour' }}
</li>
<li class="d-flex align-items-center gap-2 mb-1">
    <i class="bi bi-mortarboard text-primary"></i>
    @if ($voyage->classes_count > 0)
        {{ $voyage->classes_count }} {{ $voyage->classes_count > 1 ? 'classes' :
'classe' }}
    @else
        Aucune classe
    @endif
</li>
```

Le piège de l'issue #9 : la page d'accueil réutilise la carte, mais sa route charge les voyages sans `withCount('classes')`. `classes_count` y vaut `null` et la carte afficherait « Aucune classe ». Correction dans `routes/web.php` : `Voyage::withCount('classes')->whereDate(…)`. Compter avec `$voyage->classes->count()` fonctionnerait aussi, mais au prix d'une requête par carte, ce que le dernier critère de l'issue interdit.

Extension : annuler une fusion

`git revert` crée un nouveau commit qui défait la fusion : l'histoire de `main` reste intacte pour toute l'équipe et la règle de protection est respectée. Réécrire l'histoire (`reset` puis `push --force`) est bloqué par la règle et désynchroniserait les clones des autres.

Partie 3 — Questions

1. Après `down` puis `up`, les voyages sont toujours là : les conteneurs sont recréés, mais la base écrit dans le volume nommé `db_data`, qui n'est pas supprimé. `down -v` supprime le volume : la base repart vide, il faut relancer la migration.
2. Dans le conteneur, `127.0.0.1` désigne le conteneur `app` lui-même, où aucune base n'écoute. Compose crée un réseau où chaque service est résolu par son nom : `db` désigne le conteneur MariaDB.
3. `php:apache` suit la dernière version de PHP publiée : une construction de demain peut embarquer une version plus récente de PHP et casser l'application, alors que le code n'a pas changé. Deux postes qui construisent à des dates différentes n'obtiennent plus la même image.
4. Chaque instruction produit une couche, réutilisée si ses entrées n'ont pas changé. Tant que `composer.json` et `composer.lock` sont identiques, la couche `composer install` vient du cache : modifier une vue ne relance que `COPY . .` et la suite, en quelques secondes.
5. Par exemple : une image de base à version figée (`php:8.4-apache`, jamais `latest`) ; des dépendances installées depuis `composer.lock` commité ; un `.dockerignore` qui exclut `vendor` et `.env`, pour que rien du poste n'entre dans l'image.